

Phase 10 Structured Note: Root Functions and Root Master Shortcut

1. Current Progress Analysis

You have reached **Phase 10**, the final phase of the recurrence-relation curriculum.

Before this phase, you already learned how to read recursive code, write its recurrence relation, and solve many common recurrence patterns.

Phase	Main idea	Example recurrence	Final result
Phase 1	Turn recursive code into a recurrence relation	$T(n) = T(n - 1) + 1$	$O(n)$
Phase 2	Solve one-call decreasing recurrences	$T(n) = T(n - 1) + 1$	$O(n)$
Phase 3	Handle two decreasing recursive calls	$T(n) = 2T(n - 1) + 1$	$O(2^n)$
Phase 4	Use the decreasing master theorem	$T(n) = aT(n - b) + f(n)$	depends on a , b , and $f(n)$
Phase 5	One half-size call with constant work	$T(n) = T(n / 2) + 1$	$O(\log n)$
Phase 6	One half-size call with linear work	$T(n) = T(n / 2) + n$	$O(n)$
Phase 7	Two half-size calls with constant work	$T(n) = 2T(n / 2) + 1$	$O(n)$
Phase 8	Two half-size calls with linear work	$T(n) = 2T(n / 2) + n$	$O(n \log n)$
Phase 9	Two half-size calls with quadratic work	$T(n) = 2T(n / 2) + n^2$	$O(n^2)$

In Phases 5 to 9, the input usually changed like this:

$n \rightarrow n / 2 \rightarrow n / 4 \rightarrow n / 8 \rightarrow \dots \rightarrow 1$

That type of recurrence is called a **division recurrence**.

Now Phase 10 introduces a new type of shrinking:

$$n \rightarrow \text{sqrt}(n)$$

This is called a **root-function recurrence**.

The main recurrence in this phase is:

$$T(n) = T(\text{sqrt}(n)) + 1$$

The final result is:

$$T(n) = O(\log \log n)$$

The main lesson is:

When a recursive function repeatedly takes the square root of n , the input becomes small extremely fast.

2. Main Goal of Phase 10

The goal of Phase 10 is to understand recursive functions where the input becomes:

$$\text{sqrt}(n)$$

Instead of decreasing like this:

$$n \rightarrow n - 1$$

or dividing like this:

$$n \rightarrow n / 2$$

root functions shrink like this:

$$n \rightarrow \text{sqrt}(n)$$

The main ideas in this phase are:

1. What a root function is.
2. How to write a recurrence for root recursion.
3. Why $T(n) = T(\sqrt{n}) + 1$ gives $O(\log \log n)$.
4. What $\log \log n$ means in simple words.
5. How to use the transformation $n = 2^m$.
6. How root recurrences become division recurrences.
7. How to solve root recurrences using a root master shortcut.
8. How to solve examples such as:
9. $T(n) = T(\sqrt{n}) + 1$
10. $T(n) = 2T(\sqrt{n}) + 1$
11. $T(n) = 2T(\sqrt{n}) + \log n$

3. What Is a Root Function?

A **root function** is a recursive function where the next input is the square root of the current input.

Example recursive call:

```
fun(sqrt(n));
```

This means:

```
The next call receives sqrt(n), not n - 1 and not n / 2.
```

So if the input is very large, it becomes small very quickly.

Example:

```
65536 → 256 → 16 → 4 → 2
```

Why?

```
sqrt(65536) = 256  
sqrt(256) = 16
```

```
sqrt(16) = 4
sqrt(4) = 2
```

Only a few recursive calls happen.

4. Difference Between Decreasing, Dividing, and Root Recurrences

Type	Input change	Example	Shrinking speed
Decreasing recurrence	subtracts from n	$T(n - 1)$	slow
Dividing recurrence	divides n	$T(n / 2)$	fast
Root recurrence	takes square root of n	$T(\sqrt{n})$	very fast

Decreasing example

$$T(n) = T(n - 1) + 1$$

For $n = 16$:

$16 \rightarrow 15 \rightarrow 14 \rightarrow 13 \rightarrow \dots \rightarrow 1$

This takes about n steps.

Final time:

$$O(n)$$

Dividing example

$$T(n) = T(n / 2) + 1$$

For $n = 16$:

$16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$

This takes about $\log n$ steps.

Final time:

$O(\log n)$

Root example

$T(n) = T(\text{sqrt}(n)) + 1$

For `n = 65536`:

`65536 → 256 → 16 → 4 → 2`

This takes about `log log n` steps.

Final time:

$O(\log \log n)$

5. Main Code Example of Phase 10

```
#include <iostream>
#include <cmath>
using namespace std;

void fun(int n) {
    if (n > 2) {
        cout << n << " ";
        fun((int)sqrt(n));
    }
}

int main() {
    fun(65536);
    return 0;
}
```

6. What the Code Does

The function is:

```
void fun(int n) {
    if (n > 2) {
        cout << n << " ";
        fun((int)sqrt(n));
    }
}
```

It does three main things:

1. It checks whether `n > 2`.
2. If true, it prints the current value of `n`.
3. It calls itself again with `sqrt(n)`.

The function stops when `n` becomes `2` or smaller.

7. Why the Base Condition Is `n > 2`

In many earlier phases, the base condition was:

```
if (n > 1)
```

But in root recursion, we must be careful.

Why?

Because:

```
sqrt(1) = 1
sqrt(0) = 0
```

If the function reaches `1`, then calling `sqrt(1)` gives `1` again.

That means the function may keep calling itself forever:

```
1 → 1 → 1 → 1 → ...
```

So for square-root recursion, the safer stopping condition is:

```
if (n > 2)
```

This means the function stops when `n` becomes `2` or less.

Simple memory sentence

For root recursion, stop before the input gets stuck at `1`.

8. Breaking the Code Into Parts

Look at the important part of the code:

```
if (n > 2) {  
    cout << n << " ";  
    fun((int)sqrt(n));  
}
```

Inside each working call, there are three important parts.

Part 1: Base condition

```
if (n > 2)
```

This means the function continues only while `n` is greater than `2`.

When `n = 2`, the condition becomes false:

```
2 > 2 is false
```

So the function stops.

The base case is:

```
T(2) = 1
```

This means the stopping case takes constant time.

Part 2: Current work

```
cout << n << " ";
```

This prints one value.

Printing one value takes constant time.

So the current work is:

+1

Part 3: Recursive call

```
fun((int)sqrt(n));
```

The function calls itself once.

The input changes from:

n

to:

sqrt(n)

So the recursive part is:

$T(\text{sqrt}(n))$

9. Writing the Recurrence Relation

The total time is:

recursive call time + current work

The recursive call time is:

$T(\text{sqrt}(n))$

The current work is:

1

So the recurrence is:

$$T(n) = T(\text{sqrt}(n)) + 1$$

Since:

$$\text{sqrt}(n) = n^{(1/2)}$$

we can also write:

$$T(n) = T(n^{(1/2)}) + 1$$

Base case:

$$T(2) = 1$$

Full recurrence:

$$\begin{aligned} T(n) &= T(\text{sqrt}(n)) + 1, \text{ for } n > 2 \\ T(2) &= 1 \end{aligned}$$

10. Meaning of Each Part of the Recurrence

The recurrence is:

$$T(n) = T(\text{sqrt}(n)) + 1$$

Part	Meaning
$T(n)$	Time taken for input size n
$T(\text{sqrt}(n))$	Time taken by the recursive call on square-root input
$+1$	Constant work done in the current call
$T(2) = 1$	Base case takes constant time

In simple words:

The function does one small job now, then solves the same problem again for `sqrt(n)`.

11. Dry Run of `fun(65536)`

Call:

```
fun(65536);
```

Step-by-step table:

Call	Condition	Work done	Next call
<code>fun(65536)</code>	<code>65536 > 2</code> true	print <code>65536</code>	<code>fun(256)</code>
<code>fun(256)</code>	<code>256 > 2</code> true	print <code>256</code>	<code>fun(16)</code>
<code>fun(16)</code>	<code>16 > 2</code> true	print <code>16</code>	<code>fun(4)</code>
<code>fun(4)</code>	<code>4 > 2</code> true	print <code>4</code>	<code>fun(2)</code>
<code>fun(2)</code>	<code>2 > 2</code> false	stop	no call

Output:

```
65536 256 16 4
```

The working calls are:

```
fun(65536)
fun(256)
fun(16)
fun(4)
```

There are only 4 working calls.

This is why root recursion is very fast.

12. Why Square Root Shrinks So Fast

Compare these two shrinking patterns.

Dividing by 2

For $n = 65536$:

$65536 \rightarrow 32768 \rightarrow 16384 \rightarrow 8192 \rightarrow 4096 \rightarrow \dots \rightarrow 1$

This takes many steps.

The number of steps is:

$$\log_2(65536) = 16$$

Taking square root

For $n = 65536$:

$65536 \rightarrow 256 \rightarrow 16 \rightarrow 4 \rightarrow 2$

This takes only 4 steps.

So root recursion is even faster than normal division recursion.

13. Solving $T(n) = T(\text{sqrt}(n)) + 1$ by Substitution

We solve:

$$T(n) = T(\text{sqrt}(n)) + 1$$

Since:

$$\text{sqrt}(n) = n^{(1/2)}$$

we write:

$$T(n) = T(n^{(1/2)}) + 1$$

Step 1: Start with the recurrence

$$T(n) = T(n^{(1/2)}) + 1$$

Step 2: Substitute the smaller recurrence

The smaller input is:

$$n^{(1/2)}$$

Apply the same rule again:

$$T(n^{(1/2)}) = T((n^{(1/2)})^{(1/2)}) + 1$$

Use exponent multiplication:

$$(n^{(1/2)})^{(1/2)} = n^{(1/4)}$$

So:

$$T(n^{(1/2)}) = T(n^{(1/4)}) + 1$$

Substitute into the original recurrence:

$$T(n) = [T(n^{(1/4)}) + 1] + 1$$

So:

$$T(n) = T(n^{(1/4)}) + 2$$

Step 3: Substitute again

Apply the same rule:

$$T(n^{(1/4)}) = T((n^{(1/4)})^{(1/2)}) + 1$$

Since:

$$(n^{(1/4)})^{(1/2)} = n^{(1/8)}$$

we get:

$$T(n) = T(n^{(1/8)}) + 3$$

Step 4: See the pattern

After repeated substitution:

Step	Expression
Start	$T(n) = T(n^{(1/2)}) + 1$
After 2 steps	$T(n) = T(n^{(1/4)}) + 2$
After 3 steps	$T(n) = T(n^{(1/8)}) + 3$
After k steps	$T(n) = T(n^{(1/2^k)}) + k$

The pattern is:

$$T(n) = T(n^{(1/2^k)}) + k$$

Now we need to find k .

14. Why We Use the Transformation $n = 2^m$

The expression:

$$n^{(1/2^k)}$$

can be difficult for beginners.

So we use a helpful transformation:

$$n = 2^m$$

This also means:

$$m = \log_2(n)$$

Why is this useful?

Because square root becomes division in the exponent.

Look:

$$\text{sqrt}(n) = \text{sqrt}(2^m)$$

Since:

$$\text{sqrt}(2^m) = (2^m)^{(1/2)}$$

we get:

$$\text{sqrt}(2^m) = 2^{(m/2)}$$

So taking the square root of n becomes dividing m by 2 .

This is the key trick in Phase 10.

Simple memory sentence

When $n = 2^m$, square root of n becomes half of m .

15. Reaching the Base Case

The substitution pattern is:

$$T(n) = T(n^{(1/2^k)}) + k$$

Use:

$$n = 2^m$$

Then:

$$n^{(1/2^k)} = (2^m)^{(1/2^k)}$$

Using exponent rules:

$$(2^m)^{(1/2^k)} = 2^{(m/2^k)}$$

The recursion stops when the input becomes about **2**:

$$2^{(m/2^k)} = 2$$

For this to be true, the exponent must be **1**:

$$m / 2^k = 1$$

Now solve for **k**:

$$m / 2^k = 1$$

Multiply both sides by **2^k** :

$$m = 2^k$$

So:

$$2^k = m$$

Take logarithm base 2:

$$k = \log_2(m)$$

But:

$$m = \log_2(n)$$

Therefore:

$$k = \log_2(\log_2(n))$$

So the number of recursive levels is:

$$\log \log n$$

16. Final Solution for $T(n) = T(\text{sqrt}(n)) + 1$

Start:

$$T(n) = T(\text{sqrt}(n)) + 1$$

Rewrite square root:

$$T(n) = T(n^{(1/2)}) + 1$$

Substitute:

$$T(n) = T(n^{(1/4)}) + 2$$

Substitute again:

$$T(n) = T(n^{(1/8)}) + 3$$

Pattern:

$$T(n) = T(n^{(1/2^k)}) + k$$

Use:

$$n = 2^m$$

Then:

$$n^{(1/2^k)} = 2^{(m/2^k)}$$

Base case:

$$2^{(m/2^k)} = 2$$

So:

$$m / 2^k = 1$$

Therefore:

$$2^k = m$$

So:

$$k = \log_2(m)$$

Since:

$$m = \log_2(n)$$

we get:

$$k = \log_2(\log_2(n))$$

Final Big O:

$$T(n) = O(\log \log n)$$

17. Beginner-Friendly Meaning of $\log \log n$

You already learned that:

$\log n$

means:

How many times can we divide n by 2 until we reach 1?

Example:

$16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$

That takes 4 divisions.

Now:

$\log \log n$

means something even smaller:

How many times can we take the square root of n until we reach a small number?

Example:

$65536 \rightarrow 256 \rightarrow 16 \rightarrow 4 \rightarrow 2$

That takes only 4 root steps.

So:

$O(\log \log n)$

is very small and very fast.

Simple meaning

$\log \log n$ grows slower than $\log n$.

So:

$O(\log \log n)$ is faster than $O(\log n)$

18. Root Master Shortcut

The root master shortcut is a method for solving root recurrences quickly.

It is based on changing:

$$n = 2^m$$

Then:

$$\text{sqrt}(n) = \text{sqrt}(2^m) = 2^{(m/2)}$$

So a root recurrence in n becomes a division recurrence in m .

Root shortcut steps

When you see a recurrence with $\text{sqrt}(n)$, follow these steps:

1. Let:

$$n = 2^m$$

1. Therefore:

$$m = \log n$$

1. Rewrite $T(n)$ as:

$$T(2^m)$$

1. Rename:

$$T(2^m) = S(m)$$

1. Convert the root recurrence into a division recurrence.

2. Solve the new recurrence in terms of m .

3. Convert back using:

$$m = \log n$$

19. Why the Root Shortcut Works

The key rule is:

$$\text{sqrt}(2^m) = 2^{(m/2)}$$

This means:

$$T(\text{sqrt}(n))$$

becomes:

$$S(m / 2)$$

So a recurrence like this:

$$T(n) = T(\text{sqrt}(n)) + 1$$

becomes this:

$$S(m) = S(m / 2) + 1$$

That is now a normal division recurrence.

This is useful because you already learned how to solve division recurrences in earlier phases.

20. Example 1: $T(n) = T(\text{sqrt}(n)) + 1$

Step 1: Start with the recurrence

$$T(n) = T(\text{sqrt}(n)) + 1$$

Step 2: Let $n = 2^m$

$$n = 2^m$$

So:

$$T(2^m) = T(\sqrt{2^m}) + 1$$

Step 3: Simplify the square root

$$\sqrt{2^m} = 2^{(m/2)}$$

So:

$$T(2^m) = T(2^{(m/2)}) + 1$$

Step 4: Rename using $S(m)$

Let:

$$S(m) = T(2^m)$$

Then:

$$S(m/2) = T(2^{(m/2)})$$

So the recurrence becomes:

$$S(m) = S(m/2) + 1$$

Step 5: Solve the new recurrence

This is the same pattern as:

$$T(n) = T(n/2) + 1$$

From Phase 5, this gives:

$$S(m) = O(\log m)$$

Step 6: Convert back to n

Since:

$$m = \log n$$

we get:

$$\log m = \log(\log n)$$

Therefore:

$$T(n) = O(\log \log n)$$

Final answer

$$T(n) = T(\sqrt{n}) + 1 = O(\log \log n)$$

21. Example 2: $T(n) = 2T(\sqrt{n}) + 1$

This example has two recursive calls.

Step 1: Start with the recurrence

$$T(n) = 2T(\sqrt{n}) + 1$$

Step 2: Let $n = 2^m$

$$T(2^m) = 2T(\sqrt{2^m}) + 1$$

Step 3: Simplify the square root

$$\sqrt{2^m} = 2^{(m/2)}$$

So:

$$T(2^m) = 2T(2^{(m/2)}) + 1$$

Step 4: Rename using $S(m)$

Let:

$$S(m) = T(2^m)$$

Then:

$$S(m) = 2S(m/2) + 1$$

Step 5: Solve the new recurrence

This is the same pattern as Phase 7:

$$S(m) = 2S(m/2) + 1$$

From Phase 7:

$$S(m) = O(m)$$

Step 6: Convert back to n

Since:

$$m = \log n$$

we get:

$$T(n) = O(\log n)$$

Final answer

$$T(n) = 2T(\sqrt{n}) + 1 = O(\log n)$$

Simple meaning

The recurrence creates two root-size calls, but the root shrinking is very strong.

So the answer is not exponential.

It is only:

$$O(\log n)$$

22. Example 3: $T(n) = 2T(\sqrt{n}) + \log n$

This example has:

- two recursive calls,
- each call receives $\sqrt{n}$,
- extra work is $\log n$.

Step 1: Start with the recurrence

$$T(n) = 2T(\sqrt{n}) + \log n$$

Step 2: Let $n = 2^m$

$$T(2^m) = 2T(\sqrt{2^m}) + \log(2^m)$$

Step 3: Simplify each part

First:

$$\text{sqrt}(2^m) = 2^{(m/2)}$$

Second:

$$\log(2^m) = m$$

So:

$$T(2^m) = 2T(2^{(m/2)}) + m$$

Step 4: Rename using $S(m)$

Let:

$$S(m) = T(2^m)$$

Then:

$$S(m) = 2S(m/2) + m$$

Step 5: Solve the new recurrence

This is the same pattern as Phase 8:

$$S(m) = 2S(m/2) + m$$

From Phase 8:

$$S(m) = O(m \log m)$$

Step 6: Convert back to n

Since:

$$m = \log n$$

and:

$$\log m = \log(\log n)$$

we get:

$$T(n) = O(\log n \cdot \log \log n)$$

Final answer

$$T(n) = 2T(\sqrt{n}) + \log n = O(\log n \log \log n)$$

23. Important Memory Table for Phase 10

Original recurrence	After $n = 2^m$ transformation	Known phase pattern	Final time
$T(n) = T(\sqrt{n}) + 1$	$S(m) = S(m/2) + 1$	Phase 5	$O(\log \log n)$
$T(n) = 2T(\sqrt{n}) + 1$	$S(m) = 2S(m/2) + 1$	Phase 7	$O(\log n)$
$T(n) = 2T(\sqrt{n}) + \log n$	$S(m) = 2S(m/2) + m$	Phase 8	$O(\log n \log \log n)$

24. Time Complexity and Stack Space

For $T(n) = T(\sqrt{n}) + 1$

There is only one recursive call.

The call chain is:

$$n \rightarrow \sqrt{n} \rightarrow \text{fourth-root}(n) \rightarrow \text{eighth-root}(n) \rightarrow \dots \rightarrow 2$$

The number of levels is:

$$\log \log n$$

Each level does constant work.

So:

$$\text{Time complexity} = O(\log \log n)$$

The recursion stack also follows one chain of calls.

So:

$$\text{Stack space} = O(\log \log n)$$

For $T(n) = 2T(\sqrt{n}) + 1$

There are two recursive calls, but the root transformation changes the recurrence into:

$$S(m) = 2S(m/2) + 1$$

This gives:

$$\text{Time complexity} = O(\log n)$$

The longest path still follows repeated square roots.

So the stack depth is:

$$O(\log \log n)$$

For $T(n) = 2T(\sqrt{n}) + \log n$

The transformed recurrence is:

$$S(m) = 2S(m/2) + m$$

This gives:

$$\text{Time complexity} = O(\log n \log \log n)$$

The stack depth is still based on repeated square roots:

$$O(\log \log n)$$

25. Comparison With Earlier Phases

Recurrence	Type	Final time
$T(n) = T(n - 1) + 1$	decreasing one-call	$O(n)$
$T(n) = 2T(n - 1) + 1$	decreasing two-call	$O(2^n)$
$T(n) = T(n / 2) + 1$	dividing one-call	$O(\log n)$
$T(n) = T(\sqrt{n}) + 1$	root one-call	$O(\log \log n)$

Important lesson

The way the input changes matters a lot.

Two recurrences can look similar but have very different answers.

Compare:

$$\begin{aligned} T(n) &= T(n / 2) + 1 && \rightarrow O(\log n) \\ T(n) &= T(\sqrt{n}) + 1 && \rightarrow O(\log \log n) \end{aligned}$$

Both have one recursive call and constant work.

But square root shrinks faster than division by 2.

So root recursion is faster.

26. Why $T(n) = T(\sqrt{n}) + 1$ Is Not $O(\log n)$

A common beginner mistake is to say:

$$T(n) = T(\sqrt{n}) + 1 \text{ gives } O(\log n)$$

This is wrong.

$O(\log n)$ happens when we repeatedly divide by 2:

$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots \rightarrow 1$

But root recursion does this:

$n \rightarrow \text{sqrt}(n) \rightarrow \text{fourth-root}(n) \rightarrow \text{eighth-root}(n) \rightarrow \dots$

This shrinks even faster.

So the number of levels is:

$\log \log n$

Correct answer:

$T(n) = T(\text{sqrt}(n)) + 1 = O(\log \log n)$

27. Why $T(n) = 2T(\text{sqrt}(n)) + 1$ Is Not Exponential

Another common beginner mistake is to say:

There are two recursive calls, so it must be exponential.

This is not always true.

Two recursive calls become exponential when the input decreases slowly.

Example:

$T(n) = 2T(n - 1) + 1$

This is exponential because the input path is long:

$n \rightarrow n - 1 \rightarrow n - 2 \rightarrow n - 3 \rightarrow \dots \rightarrow 0$

But in Phase 10:

$$T(n) = 2T(\sqrt{n}) + 1$$

The input shrinks very fast:

$$n \rightarrow \sqrt{n} \rightarrow \text{fourth-root}(n) \rightarrow \dots$$

After transformation, it becomes:

$$S(m) = 2S(m/2) + 1$$

That gives:

$$O(m)$$

Since:

$$m = \log n$$

The final answer is:

$$O(\log n)$$

28. Why $\log n$ Becomes m

When we use the transformation:

$$n = 2^m$$

then:

$$\log_2(n) = m$$

Why?

Because:

$$\log_2(2^m) = m$$

So in a recurrence like:

$$T(n) = 2T(\sqrt{n}) + \log n$$

when we replace n by 2^m , the $\log n$ part becomes:

$$\log(2^m) = m$$

That is why:

$$T(n) = 2T(\sqrt{n}) + \log n$$

becomes:

$$S(m) = 2S(m/2) + m$$

29. Step-by-Step Checklist for Solving Root Recurrences

When you see a recurrence with $\sqrt{n}$, use this checklist.

Step 1: Identify that it is a root recurrence

Look for:

$$T(\sqrt{n})$$

or:

$$T(n^{1/2})$$

Step 2: Let $n = 2^m$

Write:

$$n = 2^m$$

and remember:

$$m = \log n$$

Step 3: Convert the square root

Use:

$$\text{sqrt}(2^m) = 2^{(m/2)}$$

Step 4: Rename $T(2^m)$ as $S(m)$

Write:

$$S(m) = T(2^m)$$

Then:

$$T(2^{(m/2)}) = S(m/2)$$

Step 5: Write the new recurrence in m

Examples:

$$T(n) = T(\text{sqrt}(n)) + 1$$

becomes:

$$S(m) = S(m/2) + 1$$

and:

$$T(n) = 2T(\text{sqrt}(n)) + \log n$$

becomes:

$$S(m) = 2S(m/2) + m$$

Step 6: Solve the new recurrence

Use what you learned from division recurrences.

Examples:

$$\begin{aligned} S(m) &= S(m/2) + 1 && \rightarrow O(\log m) \\ S(m) &= 2S(m/2) + 1 && \rightarrow O(m) \\ S(m) &= 2S(m/2) + m && \rightarrow O(m \log m) \end{aligned}$$

Step 7: Convert back to n

Use:

$$m = \log n$$

So:

$$\begin{aligned} O(\log m) &= O(\log \log n) \\ O(m) &= O(\log n) \\ O(m \log m) &= O(\log n \log \log n) \end{aligned}$$

30. Common Beginner Mistakes

Mistake 1: Treating $\text{sqrt}(n)$ like $n / 2$

Wrong:

$$T(n) = T(\text{sqrt}(n)) + 1 = O(\log n)$$

Correct:

$$T(n) = T(\text{sqrt}(n)) + 1 = O(\log \log n)$$

Why?

Because square root shrinks faster than dividing by 2.

Mistake 2: Forgetting to convert back from `m` to `n`

When you use:

$$n = 2^m$$

your answer is first in terms of `m`.

But the original recurrence is in terms of `n`.

So you must convert back using:

$$m = \log n$$

Example:

$$S(m) = O(m)$$

means:

$$T(n) = O(\log n)$$

not:

$$T(n) = O(n)$$

Mistake 3: Using the wrong base condition

Wrong idea:

```
if (n > 1)
```

This can be dangerous in root recursion because:

```
sqrt(1) = 1
```

So the function can get stuck.

Better idea:

```
if (n > 2)
```

This avoids getting stuck at 1.

Mistake 4: Forgetting the extra work

Example:

```
T(n) = 2T(sqrt(n)) + log n
```

The `log n` part matters.

After using:

```
n = 2^m
```

we get:

```
log n = m
```

So the transformed recurrence is:

```
S(m) = 2S(m/2) + m
```

not:

```
S(m) = 2S(m/2) + 1
```

Mistake 5: Thinking two recursive calls always means exponential time

Two recursive calls can be exponential, but only when the input does not shrink fast enough.

Example:

$$T(n) = 2T(n - 1) + 1 = O(2^n)$$

But:

$$T(n) = 2T(\sqrt{n}) + 1 = O(\log n)$$

The difference is the input size.

`sqrt(n)` shrinks much faster than `n - 1`.

31. Important Terms in Phase 10

Root function

A recursive function where the input becomes the square root of the previous input.

Example:

```
fun(sqrt(n));
```

Root recurrence

A recurrence relation where the recursive term contains `sqrt(n)`.

Example:

$$T(n) = T(\sqrt{n}) + 1$$

Square-root shrinking

The process where input changes like this:

$n \rightarrow \text{sqrt}(n) \rightarrow \text{fourth-root}(n) \rightarrow \text{eighth-root}(n) \rightarrow \dots$

Double logarithm

$\log \log n$ means a logarithm applied twice.

It appears when the input repeatedly becomes $\text{sqrt}(n)$.

Example:

$$T(n) = T(\text{sqrt}(n)) + 1 = O(\log \log n)$$

Transformation $n = 2^m$

A substitution used to make root recurrences easier.

If:

$$n = 2^m$$

then:

$$m = \log n$$

This changes square-root recursion into division recursion.

$S(m)$ notation

After setting:

$$n = 2^m$$

we rename:

$$T(2^m) = S(m)$$

This helps us solve the recurrence in terms of m .

32. Complete Coding Program for Phase 10

```
#include <iostream>
#include <cmath>
using namespace std;

// This function prints n, then calls itself with sqrt(n).
void fun(int n) {
    if (n > 2) {
        cout << n << " ";           // constant work: +1
        fun((int)sqrt(n));           // recursive call: T(sqrt(n))
    }
}

int main() {
    int n = 65536;

    cout << "Output: ";
    fun(n);
    cout << endl;

    return 0;
}
```

Output

Output: 65536 256 16 4

Recurrence

$$T(n) = T(\sqrt{n}) + 1$$

Time complexity

$$O(\log \log n)$$

Stack space

$$O(\log \log n)$$

33. Practice Problems With Answers

Problem 1

Solve:

$$T(n) = T(\sqrt{n}) + 1$$

Answer:

$$O(\log \log n)$$

Reason:

Use $n = 2^m$.

Then:

$$S(m) = S(m/2) + 1$$

So:

$$S(m) = O(\log m)$$

Convert back:

$$m = \log n$$

So:

$$T(n) = O(\log \log n)$$

Problem 2

Solve:

$$T(n) = 2T(\sqrt{n}) + 1$$

Answer:

$$O(\log n)$$

Reason:

Use $n = 2^m$.

Then:

$$S(m) = 2S(m/2) + 1$$

This gives:

$$S(m) = O(m)$$

Convert back:

$$m = \log n$$

So:

$$T(n) = O(\log n)$$

Problem 3

Solve:

$$T(n) = 2T(\sqrt{n}) + \log n$$

Answer:

$$O(\log n \log \log n)$$

Reason:

Use $n = 2^m$.

Since:

$$\log n = m$$

we get:

$$S(m) = 2S(m/2) + m$$

This gives:

$$S(m) = O(m \log m)$$

Convert back:

$$\begin{aligned} m &= \log n \\ \log m &= \log \log n \end{aligned}$$

So:

$$T(n) = O(\log n \log \log n)$$

34. Final Summary of Phase 10

Phase 10 teaches **root-function recurrences**.

The main idea is that the input changes from:

n

to:

$\text{sqrt}(n)$

This makes the input very small very quickly.

The most important recurrence is:

$$T(n) = T(\text{sqrt}(n)) + 1$$

The final answer is:

$$O(\log \log n)$$

The most important trick is:

$$n = 2^m$$

because:

$$\text{sqrt}(2^m) = 2^{(m/2)}$$

So root recursion becomes division recursion.

The most important conversion is:

$$m = \log n$$

So after solving in terms of m , always convert back to n .

35. Final Memory Sentences

1. Root recursion means the next input is $\text{sqrt}(n)$.
 2. $T(n) = T(\text{sqrt}(n)) + 1$ gives $O(\log \log n)$.
 3. Use $n = 2^m$ to convert root recurrence into division recurrence.
 4. After solving in m , convert back using $m = \log n$.
 5. Square root shrinking is faster than dividing by 2.
 6. Two recursive calls are not always exponential; input shrinking matters.
 7. For root recursion, avoid getting stuck at $\text{sqrt}(1) = 1$.
-

36. One-Page Phase 10 Cheat Sheet

Main recurrence

$$T(n) = T(\text{sqrt}(n)) + 1$$

Rewrite square root

$$\text{sqrt}(n) = n^{(1/2)}$$

Substitution pattern

$$T(n) = T(n^{(1/2^k)}) + k$$

Transformation

$$\begin{aligned} n &= 2^m \\ m &= \log n \end{aligned}$$

Key identity

$$\text{sqrt}(2^m) = 2^{(m/2)}$$

Main result

$$T(n) = T(\text{sqrt}(n)) + 1 = O(\log \log n)$$

Root shortcut table

Recurrence	Transformed recurrence	Final answer
$T(n) = T(\text{sqrt}(n)) + 1$	$S(m) = S(m/2) + 1$	$O(\log \log n)$
$T(n) = 2T(\text{sqrt}(n)) + 1$	$S(m) = 2S(m/2) + 1$	$O(\log n)$
$T(n) = 2T(\text{sqrt}(n)) + \log n$	$S(m) = 2S(m/2) + m$	$O(\log n \log \log n)$

Final beginner memory line

Square-root recursion is solved by changing n into 2^m , solving the new recurrence in m , then converting back using $m = \log n$.